

SHOULD BE REPLACED ON REQUIRED TITLE PAGE

Instruction

1. Open needed docx template (folder "title"/<your department or bach if bachelor student>.docx).
2. Put Thesis topic, supervisor's and your name in appropriate places on both English and Russian languages.
3. Put current year (last row).
4. Convert it to "title.pdf," replace the existing one in the root folder.

Contents

1	Introduction	7
1.1	Background	7
1.2	PhantomOS	8
1.3	Genode	9
1.4	Problem statement	10
2	Literature Review	12
2.1	Persistence	12
2.1.1	Persistence definitions	12
2.1.2	Orthogonal persistence principles	13
2.1.3	Advantages of a persistent system	13
2.1.4	The cost of persistence	14
2.2	PhantomOS	15
2.2.1	PhantomOS Overview	15
2.2.2	Mechanisms for ensuring persistence	16
2.2.3	System and Persistent Environment features	17
2.3	OS Phantom to Genode port	18
2.3.1	Reason for porting	18
2.3.2	Affected components	19

CONTENTS	3
2.3.3 Snapper	19
2.4 Data protection at rest	19
3 Methodology	20
3.1	20
4 Implementation	21
4.1	21
4.1.1	21
5 Evaluation and Discussion	22
5.1	22
6 Conclusion	23
6.1	23
Bibliography cited	24
A Extra Stuff	27
B Even More Extra Stuff	28

List of Tables

List of Figures

Abstract

The evolution of operating systems over the decades has been shaped by a proven conceptual model that emerged in the mid-20th century with the advent of UNIX. Today, however, it is increasingly revealing fundamental limitations. Attempts to circumvent these limitations as early as the 1980s led, among other things, to the emergence of the concept of operating systems with persistent memory. Despite the merits of this paradigm, research was effectively halted as it ran into hardware limitations. Since then, computer performance has increased significantly, which has spurred new work in this direction. A striking illustration of this is the Phantom OS. However, writing and implementing a completely new operating system is a very time-consuming and labor-intensive process and is prone to a large number of errors. The use of proven mechanisms helps mitigate these consequences, a process facilitated by porting to Genode. In this paper, we examine the process of adapting PhantomOS components to Genode. We describe the difficulties encountered, the OS's shortcomings, and discuss solutions that help eliminate or mitigate these problems.

Chapter 1

Introduction

1.1 Background

Today, a widely used concept involves dividing data into two categories: active and stored. In this model, the programmer is responsible for moving data between these states. In the context of this paper, this approach will be referred to as a “two-level store.” The obvious drawbacks of this approach include:

- Data loss due to unexpected shutdowns or process termination;
- Increased development effort (and higher cost);

This approach is contrasted with the concept of orthogonal persistence, discussed by Atkinson and Morrison in the 1990 [1]. It entails that data is constantly accessible and does not require explicit actions on the part of the programmer to save it to the drive. Instead, the responsibility for this falls on the environment in which the program is executed. This reduces the amount of data manipulation, which typically accounts for up to 30% of the code [1]. The number of errors when writing such programs is also significantly reduced. One of the less obvious

advantages of this approach is the highly efficient use of disk bandwidth [2]. Over time, several operating systems have emerged that implement this concept. These include KeyKOS (EROS and Coyotos), Multics (with single-level storage), and others. All of these systems are quite old, and research in this area came to a standstill after their introduction. However, significant advances in hardware capabilities have reignited interest in this field, leading to the development of the Aurora operating system (transparent persistence) [3]. The PhantomOS project is one such modern system that implements the concept of orthogonal persistence.

1.2 PhantomOS

According to the developers themselves, Phantom OS is an operating system that implements the principles of orthogonal persistence within the Phantom Virtual Machine (PVM) [4]. According to the documentation, the system guarantees recovery even in the event of an unexpected reboot, provided the consistent data is not too old. This is achieved through the use of a snapshot mechanism: from time to time, the system saves the PVM's state to disk without interrupting its operation, which is accomplished using copy-on-write (CoW). At present, Phantom is a proof-of-concept (PoC) for the concept of orthogonal persistence. Many mechanisms and subsystems have been implemented, and their functionality has been verified to some extent, but they have not been fully tested. The current level of stability does not allow the OS to be used for industrial purposes, and work on refining it is still ongoing. One area of this work involves porting PhantomOS to the Genode framework. This should accelerate development and help avoid a large number of errors by using proven solutions, as well as positively impact system security through improved isolation, which is enabled by the use

of capability-based security.

1.3 Genode

The Genode OS Framework is a set of tools for building operating systems. It provides a model in which component resources are isolated and their interactions are mediated via RPC. The framework supports multiple kernels as a foundation (including NOVA, formally verified seL4 [5], Fiasco.OC, and Linux), and provides a ready-made set of drivers, file systems, and services, allowing developers to focus on application logic without having to reinvent or port the underlying infrastructure. Access to resources in Genode is organized in accordance with the principles of capability-based security. In this context, a capability is a token granting the right to perform a specific operation on a specific object. A component can use a resource only if it possesses the corresponding capability, which was explicitly granted to it by a parent component. At the same time, the parent component is fully responsible for granting the appropriate capabilities to the child component and for passing its requests further up the hierarchy. This prevents unauthorized access to resources by bypassing the hierarchy, which differs significantly from the approach of the operating systems we are familiar with today, where a process with sufficient privileges can access any resource. [6]. As part of the phantomuserland-snapper project [7], PhantomOS is being ported to Genode as a set of components. The Phantom Virtual Machine (PVM) runs as a Genode user process. To create, manage, and restore snapshots, snapper [8] was developed, which uses the file system for storage. This approach allows Phantom to use Genode's proven drivers and services without implementing them independently; the isolation of Genode components provides an additional layer of

protection; and the use of microkernels enables a trusted computing base (TCB) of modest size by today's standards, which also positively impacts the OS's security.

1.4 Problem statement

However, some security issues still need to be addressed. For instance, unlike most modern operating systems, which support disk encryption, PhantomOS does not support this feature in any form. A review of the literature also revealed that no research has been conducted on PhantomOS in this area. Porting does not solve the problem either, as Genode lacks ready-made, adapted disk encryption mechanisms. Moreover, the information flushed to disk at the moment a snapshot is created and stored there may be more sensitive than the files on the disk, since confidential data may be present in memory at the time the snapshot is taken. This is largely similar to the security issue with virtual machine snapshots. Under certain conditions, this could allow the state and subsequent operation of the machine to be fully reproduced on an attacker's device. This necessitates the implementation of protection for snapshots at rest. Thus, the goal of this work is to ensure the confidentiality of PhantomOS snapshots at rest. This leads to the following objectives:

- Analyze computer usage scenarios and identify among them the secure scenarios that we will support;
- Develop or adapt a Genode component that implements such a scenario;
- Verify the correctness of the protection: demonstrate that the snapshot's contents are inaccessible;

- Evaluate the impact of encryption on performance by comparing the speed of operations with encryption enabled and disabled;

Chapter 2

Literature Review

This chapter provides an overview of existing research related to our topic. The chapter is divided into four main sections, each of which will examine works pertaining to the following

Section 1: The concept of orthogonal persistence and its implementation options;

Section 2: PhantomOS and its implementation of the concept of orthogonal persistence;

Section 3: Details and features of the PhantomOS port to Genode;

Section 4: Methods of protecting data at rest and their characteristics;

2.1 Persistence

2.1.1 Persistence definitions

Data persistence refers to the period of time during which data exists and is used [1]. This is precisely how some of the earliest researchers in the

field—Morrison R. and Atkinson M. P.—define this concept. In their work, they identify categories of persistence, divided into two groups:

1. Provided by the programming language;
2. Provided by the environment;

Researchers are striving to develop systems in which data usage is independent of the group or category to which the data belongs.

2.1.2 Orthogonal persistence principles

The concept of orthogonal persistence is based on principles formulated by researchers:

1. Independence;
2. Orthogonality of data types;
3. Persistence Identification;

In this case, independence means that data persistence does not depend on how the data is manipulated (the user cannot move data between long-term and short-term storage). The orthogonality of data types ensures that there are no cases in which data of any type cannot be persisted. The third principle means that persistent object identification does not relate to the type of such an object. A system that follows all three principles is orthogonally persistent.

2.1.3 Advantages of a persistent system

The desire to create such a system stems from the fact that it offers a number of advantages over conventional systems. These include [9]:

1. Increased programming productivity through simplified semantics;
2. Avoiding ad hoc solutions for data transformation and long-term data storage;
3. Providing security mechanisms for the entire environment;
4. Support for gradual evolution;
5. Automatically preserving referential integrity across the entire computational environment for the entire lifetime of an application

All of these are, to some extent, the result of simplifying the memory management model for application programmers. This eliminates a huge amount of code that would otherwise be required to handle I/O and data storage. This also includes serialization/deserialization and initialization/deinitialization.

2.1.4 The cost of persistence

All of the advantages outlined above come at a cost. In addition to the general challenges associated with creating and implementing a new system—especially one built on principles that are entirely different from those we are accustomed to—there are also challenges specific to orthogonally persistent systems. These include:

1. The need for a stable object store: This depends largely on the specific implementation method, so we will return to this topic in Section 2.2.
2. Reduced performance of some applications: Because the programmer does not have full control over the location of the data, access speed may be reduced. A similar problem exists when using virtual memory.

3. The cost of providing language-independent binding mechanisms.

Looking ahead, I will say that Phantom has solutions to minimize each of these problems, so we will return to them in Section 2.2. For now, note that point 1 depends largely on the implementation, and it is difficult to generalize information regarding it. Problems similar to point 2 also affect (albeit often to a lesser extent) systems with virtual memory, since the programmer does not have full control over the layout of the data, which can slow down access to it.

2.2 PhantomOS

2.2.1 PhantomOS Overview

PhantomOS is an open-source operating system. The concept was conceived by Dmitry Zavalishin, who also made the primary contribution to the system's development. Like other modern operating systems, Phantom features multitasking [10], [11], [12], a windowing subsystem [13], a network stack, and other familiar attributes. However, for our work, another feature is of greatest interest. PhantomOS provides an orthogonally persistent environment for applications within the Phantom Virtual Machine (PVM), where code is executed in the Phantom programming language. However, it is worth noting that there is also a POSIX compatibility subsystem. Access via arbitrary addresses is prohibited. Phantom already implements subsystems such as:

- Kernel itself: threads, synchronization, persistent memory management;
- Bytecode virtual machine - running native applications;
- POSIX layer - runs Linux-compatible (but not yet persistent) code;

- Graphics subsystem - Windows, controls, UI;
- Networking (TCP/IP);
- Phantom language compiler - the most native userland language;
- Java to Phantom translator - work in progress;
- Python to Phantom translator - just started;

All of this is specified in the documentation [4].

2.2.2 Mechanisms for ensuring persistence

Dmitry Zavalishin believes that it is impossible to preserve the state of the entire machine, but this is not necessary to achieve persistence [14]. Therefore, we will now discuss the mechanisms for ensuring a persistent environment. Persistence in PhantomOS is achieved by page-by-page mapping of the entire memory (PVM) to disk and periodically saving it. The kernel manages the snapshot process. It takes snapshots after first bringing the programs to a state where they are fully represented in memory. That is, all information from the processor registers is also transferred for this purpose. Application programs are blocked only during this brief preparation phase. The disk is switched to read-only mode during this process, but this does not cause a lock throughout the entire process of writing the snapshot to disk thanks to the use of CoW: if a page has been written to the snapshot, nothing prevents access to it; otherwise, a copy is created, which the program operates on, and the original page is moved closer to the front of the save queue [14]. Writing the entire memory to disk with every snapshot is a very costly operation that is unnecessary. To optimize this process, only the increment

is written to disk, and preemptive writing is also used. The latter, by the way, serves two purposes:

- To speed up the snapshot writing process;
- To meet the increased demand for memory during the snapshot;

2.2.3 System and Persistent Environment features

One of the goals behind the system's design is to make life easier for programmers. They no longer need to manage memory; instead, the garbage collector handles this task. Other key features of the system include a global address space and the requirement that objects can only be accessed via a reference. Access via arbitrary addresses is prevented, which has a positive effect on security. It is in this environment that the garbage collector operates. The large amount of virtual memory and the inability to use stop-world algorithms led to a strategy of using two garbage collectors [15]:

1. Partial;
2. Full;

The first is fast, inexpensive, and possibly incomplete. It is assumed that it cleans up garbage in RAM without waiting for objects to be flushed to disk. It runs continuously and is currently implemented based on counting the number of references to an object. The second should be full, but it can run periodically thanks to the presence of the first. At the same time, a stop-world implementation is also possible due to the persistence of garbage: that is, what was garbage in the old snapshot will remain garbage in the new one. Based on this idea, it is

proposed to perform garbage collection on the old snapshot, although not the entire snapshot may be used for this, but only a reflection of its memory state. However, there are still a number of problems that need to be solved before its implementation [16].

2.3 OS Phantom to Genode port

2.3.1 Reason for porting

As previously mentioned, PhantomOS is currently being ported to the Genode framework. This is being done to improve **Antonov_Antonov**;

- Reliability;
- Security;
- Interoperability;

- In this way, we plan to achieve the required quality. One of the main reliability issues—the new kernel with built-in drivers—is planned to be resolved by using kernels supported by Genode, including many microkernels known for their high reliability. It is also proposed to replace new, insufficiently tested components with already proven ones offering the same functionality. This will also resolve the issue of interoperability, as there will be no need to rewrite existing components, and the focus can shift to the system's logic. The Genode developers have taken a thorough approach to security. The framework uses a capability-based security system. Microkernels are often used as the system kernel. This makes it possible to isolate components to a significant degree and configure access to resources

more flexibly, allowing for a minimal TCB. This results in a minimal attack surface, which has an extremely positive effect on reliability.

2.3.2 Affected components

2.3.3 Snapper

2.4 Data protection at rest

Chapter 3

Methodology

3.1

Chapter 4

Implementation

4.1

4.1.1

Chapter 5

Evaluation and Discussion

5.1

Chapter 6

Conclusion

6.1

...

Bibliography cited

- [1] R. Morrison and M. P. Atkinson, “Persistent languages and architectures,” in *Security and Persistence*, J. Rosenberg and J. L. Keedy, Eds., Springer-Verlag, 1990, pp. 9–28.
- [2] A. C. Bomberger, W. S. Frantz, A. C. Hardy, N. Hardy, C. R. Landau, and J. S. Shapiro, “The KeyKOS nanokernel architecture,” in *Proceedings of the USENIX Workshop on Microkernels and Other Kernel Architectures*, Seattle, WA, 1992, pp. 95–112.
- [3] E. Tsalapatis, R. Hancock, T. Barnes, and A. J. Mashtizadeh, “The Aurora single level store operating system,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, ACM, 2021, pp. 788–803. DOI: [10.1145/3477132.3483563](https://doi.org/10.1145/3477132.3483563)
- [4] D. Zavalishin, *Phantom os documentation*, Available: <https://app.readthedocs.org/projects/phantomdox/downloads/pdf/latest/>, 2020.
- [5] G. Klein et al., “seL4: Formal verification of an OS kernel,” in *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, Big Sky, MT, USA: ACM, 2009, pp. 207–220. DOI: [10.1145/1629575.1629596](https://doi.org/10.1145/1629575.1629596)

- [6] N. Feske, *Genode operating system framework foundations*, Available: <https://genode.org/documentation/genode-foundations-25-05.pdf>, 2025.
- [7] R. Mitov, *Phantomuserland-snapper*, Available: github.com/rumenmitov/phantomuserland-snapper.
- [8] R. Mitov, *Snapper*, Available: <https://github.com/rumenmitov/snapper>.
- [9] A. Dearle, G. N. C. Kirby, and R. Morrison, “Orthogonal persistence revisited,” *arXiv preprint*, 2010, arXiv:1006.3448. [Online]. Available: <https://arxiv.org/abs/1006.3448>
- [10] Д. Завалишин, *Делаем мультизадачность*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: <https://habr.com/ru/articles/282037/>
- [11] Д. Завалишин, *Преemptивность: Как отнять процессор*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: <https://habr.com/ru/articles/282049/>
- [12] Д. Завалишин, *От шедулера к планировщику*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: <https://habr.com/ru/articles/282213/>
- [13] Д. Завалишин, *Ос фантом: Оконная подсистема — делаем контролы*, Хабр, Дата обращения: 2025, Nov. 2019. [Online]. Available: <https://habr.com/ru/articles/472160/>
- [14] Д. Завалишин, *Персистентная оперативная память*, Хабр, Дата обращения: 2025, Mar. 2016. [Online]. Available: <https://habr.com/ru/articles/279443/>

-
- [15] Д. Завалишин, *Сборка мусора в персистентной модели: От терабайта и дальше*, Хабр, Дата обращения: 2025, Apr. 2016. [Online]. Available: <https://habr.com/ru/articles/281236/>
- [16] Д. Завалишин, *Фантом: Большая сборка мусора*, Хабр, Дата обращения: 2025, Jun. 2017. [Online]. Available: <https://habr.com/ru/articles/331974/>

Appendix A

Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Appendix B

Even More Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.